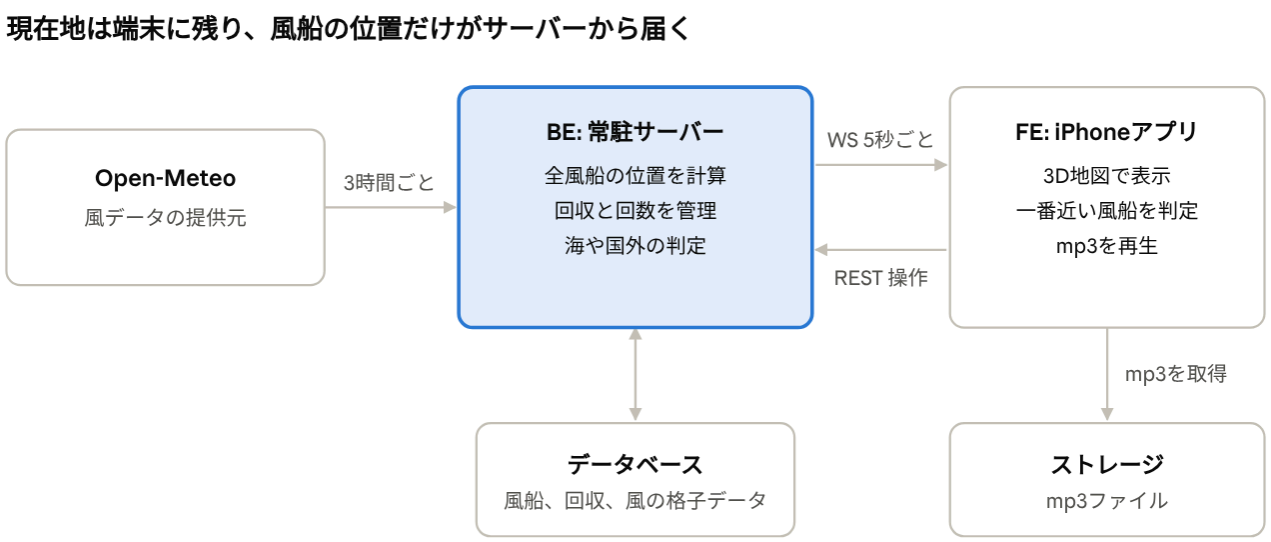
推し風（仮）設計書

Nov 1, 2026 · @tomoemon83

概要

推し風は、タイムラインでは埋もれる推し曲を、実際の風に乗せて日本中へ届け続けるiOSアプリです。ユーザーは推し曲を載せた風船を1つ飛ばし、自分の近くを漂う他人の風船の曲を聴きます。

項目	決定
サービス名	推し風（仮）
ハッカソンのテーマ	オープンデータ × 推し
掛け算	風データ × 音楽 × 風船。風データがないと風船が動かず、サービスが成り立たない
目標	面白いものを作る、新しい技術に挑戦する
挑戦する技術	3D地図、リアルタイム通信
開発期間	1週間
開発体制	AI開発。人数は1人と仮定
発表	iPhone実機2台以上でデモ
収益	考えない



システム構成·FEとBE、5つの要素と主な通信

事業設計

核は、聴く側の偶然の出会いと、飛ばす側の布教の手応えです。流された回数と回収の通知が、飛ばす人の報酬になります。

課題と対象

- **課題（仮説）**: 反響がない投稿は、すぐに埋もれてしまう
- **対応**: 風船は反応がなくても空に残り、次の街の誰かへ届き続ける
- **サービスの対象**: 全規模の推し曲
- **発表の主人公（仮置き）**: 周りに知っている人が少ない推しを持つ、20代の営業職

データと音源

- **風データ**: Open-Meteo。観測の生中継ではなく気象モデルの予報値なので、発表では「最新の気象データ」と説明する
- **音源**: ユーザーがmp3をアップロードする

風船のルール

項目	決定
載せる曲	1個につき1曲
同時に飛ばせる数	1人1個

項目	決定
出発地点	現在地を端末でぼかしてから送る。生の現在地はサーバーへ送らない
速度	実際の風と等倍を初期値にする。倍率はサーバー全体で1つ持ち、デモでは早送りする
対象の風船	全ユーザーの風船を区別なく流す。自分の風船と割った風船は除く
流された回数	再生開始から5秒で1回。同じ人は同じ風船で1回まで
終わり方	回収される、または海や国外へ出る。時間切れと自分で割る操作はなし
旅の終わり	飛ばした人に旅の記録カードを届ける

回収と割る

操作	回収	割る
意味	今日の1曲として保存する	自分の世界から消す
風船への影響	全員の世界から消える	他の人には影響しない
回数制限	1日1回	なし
飛ばした人への通知	あり	なし
その後	お気に入りに保存される	その人には二度と流れない

FE設計（アプリ）

FEはiOSネイティブのアプリで、画面、再生、一番近い風船の判定、ローカル通知を担当します。

メイン画面は3D地図の全画面で、地図、再生バー、飛ばすボタンの3つに絞ります。地図の閲覧はログインなしでできます。

画面一覧

画面	内容
初回説明	スライド3枚、スキップ可能。時間が余れば3D地図上の実演を足す

画面	内容
メイン	3D地図の全画面、下部の再生バー、飛ばすボタン、右上のマイページアイコン
ログイン	メールとパスワード。飛ばす、回収、流すの操作時に求める
曲の選択	自分がアップロードした曲の一覧。ファイルアプリからmp3を選んで追加できる
自分の風船	地図が自分の風船へ移動し、現在地、曲、回数を表示する
マイページ	お気に入りと旅の記録カードをタブで切り替える
設定	流す／流さないの切り替え（初期値はOFF）、ログアウト

操作

- **飛ばすボタン**: 風船が飛行中は「🎈12回」のようなボタンに変わり、押すと自分の風船へ移動する
- **割る**: 地図上の風船をタップするか、アプリを開いた状態でスマホを振る
- **回収**: 操作方法は保留。当面は再生バーのボタンとする

再生の挙動

- 流す設定がONの間は、起動直後から流れ、バックグラウンドでも流れ続ける
- 風船が0個の間は無音ループを鳴らし、アプリと通信を止めない
- 最寄りが同じまま曲が終わったら、同じ曲をループする
- イヤホンが外れたときや、着信などで音声が中断されたときは、再生を一時停止する
- イヤホンでの一時停止は、アプリかロック画面の操作で再開する。着信などの割り込みは、終了時に再開する
- 一番近い風船が変わったら、フェードで曲を切り替える
- 切り替えの判定は30秒に1回とし、今の風船より1割以上近いときだけ切り替える

通知

Developer Programに加入しないので、APNsは使いません。再生中にサーバーから受信し、端末内のローカル通知を出す設計です。再生していないときは届きません。

アプリを開いている間も通知を表示します。切断中に届かなかった結果は、起動時に `/me/journeys` で取り戻します。

端末内の処理

- **最寄り判定**: 届いた全風船の位置と現在地を比べて、一番近い風船を決める。自分の風船は `/me/balloons` のID、割った風船は `/me/popped` のIDで除く
- **位置の補間**: 5秒ごとの配信の間は、位置と速度から地図上の風船を動かす
- **回数の送信**: 実際に5秒再生されたら `POST /balloons/:id/plays` を送る。失敗したら再送する
- **回数の更新**: 前面にある間は30秒ごとに `/me/balloons` を読み、ボタンの回数を更新する
- **現在地**: BEへ送らない。送るのは、端末でぼかした出発地点と、逆ジオコーディングした地名だけ
- **mp3**: 初めて見る曲IDは `/tracks/:id` で読んでキャッシュし、URLからストレージへ取りに行く
- **同期**: 起動、復帰、再接続のときに `/me/balloons`、`/me/popped`、`/me/journeys` を読み直す
- **再接続**: 指数バックオフで再接続する。`ping` が途切れたら切断とみなす
- **トークン**: Keychainに保存する。401ならトークンを消してログイン画面へ戻す

権限

- **位置情報**: 使用中のみの許可を求める。拒否されたらアプリを使えないので、許可を求める案内画面を出し、設定アプリへ誘導する。バックグラウンドの位置更新は、1日目の検証で必要と分かったら追加する
- **通知**: 流すをONにしたときに許可を求める

BE設計（サーバー）

BEはRailsで作り、REST API、WebSocket、定期処理を持ちます。データベースはPostgreSQLで、mp3のストレージはBEの外に置きます。

定期処理のプロセスが全風船の位置を計算し、5秒ごとにWebSocketで全端末へ配信します。一番近い風船は端末で判定するので、ユーザーの現在地はサーバーへ送りません。サーバーが受け取る位置は、風船の出発地点だけです。

構成

項目	決定
サーバー	Rails（APIモード）。Webと定期処理の2プロセスを、常時起動できる環境で動かす
データベース	PostgreSQL

項目	決定
WebSocket	Action Cable。`/ws` に置き、Origin検査を解除する。配信はSolid CableをPostgreSQL上で使い、メッセージの保持は数分にする
ストレージ	BEの外に置く。アップロードされたmp3はBEが保存し、アプリは曲のURLから直接取得する
風データの取得	日本周辺の1度格子（約830地点）を3時間ごとに、300地点ずつ3回に分けて取得する。予報も有効時刻付きで保存し、失敗したら前回の予報で動かす。地点の間は補間する
位置の配信	5秒ごと。間は端末で補間する
定期処理	advisory lockで1プロセスだけを動かす。例外が起きても続け、前回が終わっていなければ次を飛ばす。位置は実測の経過時間で計算する
認証	メールとパスワード。パスワードはハッシュ化して保存する。ログイン時にランダムなトークンを発行し、ハッシュ化して保存する。有効期限は設けない。メールの実在確認とパスワード再設定は作らない

REST API

メソッド	パス	用途
POST	`/auth/signup`	新規登録
POST	`/auth/login`	ログイン
DELETE	`/auth/session`	ログアウト
GET	`/me`	ログイン中の利用者
POST	`/tracks`	曲をアップロードする（mp3、曲名、アーティスト名）
GET	`/tracks/:id`	曲名、アーティスト名、mp3のURL。認証は不要
GET	`/me/tracks`	自分がアップロードした曲の一覧
POST	`/balloons`	風船を飛ばす（自分の曲ID、ぼかした出発地点、地名）
GET	`/me/balloons`	飛行中の自分の風船と回数

メソッド	パス	用途
POST	`/balloons/:id/plays`	5秒再生されたら回数を加算（1人1回）
POST	`/balloons/:id/collect`	回収（1日1回）
POST	`/balloons/:id/pop`	割る（自分の世界からのみ消す）
GET	`/me/popped`	割った風船のID一覧
GET	`/me/favorites`	お気に入り一覧
GET	`/me/journeys`	旅の記録カード一覧

エラーは `{code, message}` の形で返します。

API	応答	条件
`POST /tracks`	422	mp3ではない、または20MBを超えている
`POST /balloons`	403	他の人の曲を飛ばそうとした
`POST /balloons`	409	飛行中の風船がある。アプリは `/me/balloons` を読み直す
`POST /balloons`	422	出発地点が対象範囲の外にある
`POST /balloons/:id/collect`	200	回収済みの本人が再送した
`POST /balloons/:id/collect`	403	自分の風船を回収しようとした
`POST /balloons/:id/collect`	409	飛行中ではない。先着に負けた場合も含む
`POST /balloons/:id/collect`	429	今日すでに回収している
`POST /balloons/:id/plays`	200	再送や終了済みの風船でも成功とし、回数は増やさない
`POST /balloons/:id/pop`	200	再送しても成功とする
トークンが必要なAPI	401	トークンがない、または無効

WebSocket `/ws`

サーバーから端末への配信専用です。接続は匿名で許可するので、未ログインでも閲覧できます。全員向けのチャンネルと、トークンが必要な本人用チャンネルに分けます。ログイン後とログアウト後は再接続します。

イベント	届く相手	タイミング
`balloons.snapshot`	全員	接続時と5秒ごと
`balloon.removed`	全員	回収された、または海や国外へ出たとき
`balloon.collected`	飛ばした人	自分の風船が回収されたとき
`balloon.ended`	飛ばした人	旅が終わり、記録カードができたとき

`balloons.snapshot` が地図の状態の正で、新しい風船も次の配信で届きます。`owner_id` は載せません。`balloon.removed` で消したIDは、古い配信で復活しないよう少しの間無視します。

```
{"seq": 1, "server_time": "...", "speed_scale": 1,
 "balloons": [{"id": "...", "track_id": "...", "lat": 0, "lng": 0, "ve": 0,
"vn": 0}]}
```

業務ルール

- **同時回収**: 先着1人を成功とし、もう1人には409を返す。その人の回収回数は消費しない
- **回収の処理**: 1トランザクションで、`flying` の風船だけを `collected` へ更新する。更新できたらお気に入りと記録カードを作る。今日の回収と重なったら429を返し、ロールバックして風船は消さない
- **1日の区切り**: 日本時間の0時
- **回収の上限**: 今日すでに回収していれば、429を返す
- **飛ばす制限**: 飛行中の風船があれば、409を返す。飛ばせるのは自分の曲だけ
- **ログイン**: 全員向けの閲覧と `/tracks/:id` 以外の機能は、トークンを要求する
- **風船の状態**: `status` は `flying`、`collected`、`lost` の3つ。記録カードは `journeys` の行で表す
- **定期処理の更新**: 位置と速度の列だけを、`flying` の行に限って更新する。終了処理も更新できたときだけ記録カードを作る
- **回数の加算**: `play_count` はSQLの増分で更新する
- **海や国外の判定**: 日本の陸地マスク（0.05度程度）の外へ出たら割れる。範囲を超えた場合も割れる
- **地名**: 出発地は端末の逆ジオコーディング、終わった場所は海域の矩形テーブルで決める

- **管理者操作**: デモ前に全風船の消去と、ボット風船20〜50個の投入を `rake` で行う。自分の風船は再生されないので、切り替えを見せるのに必要になる

未決事項とリスク

最大のリスクは、バックグラウンド動作が未確認なことです。機能を作り始める前に、実機で検証します。

1日目の検証

- [ ] 画面を閉じて次の曲へ続けて再生できるか確認する
- [ ] 再生中にWebSocketを受信し続けられるか確認する
- [ ] 受信をきっかけにローカル通知を出せるか確認する
- [ ] 風船0個の無音状態を5分続けても動くか確認する
- [ ] 曲の終了、イヤホンを外して30秒後の再開、着信を確認する
- [ ] 無料アカウントでBackground Modesを有効にしてビルドできるか確認する
- [ ] 使う台数の実機にインストールできるか確認する
- [ ] 本番URLのAction Cableへ実機から接続できるか確認する

未決事項

- [ ] 回収の操作方法を決める
- [ ] 陸地マスクの元データを決める
- [ ] 3D地図の実現方法を2日目までに決める（MapKitを第一候補として試す）
- [ ] BEの配置先を決める（常時起動で2プロセスが動くこと。Renderの無料枠は使えない）
- [ ] ストレージの種類を決める
- [ ] サービス名を確定する

リスク

リスク	影響	対応
バックグラウンド動作が想定通りに動かない	聴く体験と通知の設計を見直す	1日目に実機で検証する
Open-Meteoの無料枠と規約	無料枠は1分600回、1時間5,000回、1日1万回未満、月30万回。複数地点の加重は不明確で、共有ホストでは制限に早く達する恐れがある	分割して取得する。非商用の範囲で使い、CC-BY 4.0に従ってクレジットを表示する

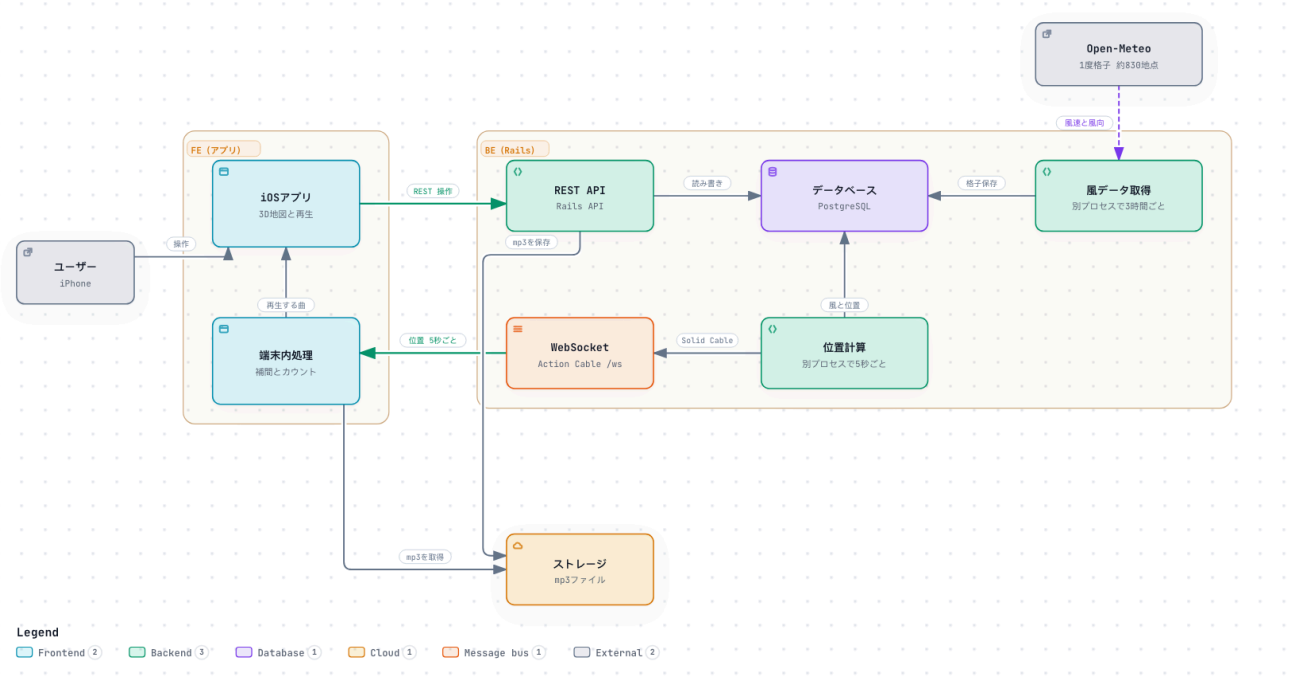
リスク	影響	対応
市販曲のmp3の権利	不特定の人への配信は権利侵害になる恐れがある	ハッカソンでは割り切る。公開範囲を広げるなら見直す
実機の7日期限	発表中にアプリが起動できなくなる	前日に入れ直して動作を確認し、当日はMacとケーブルを持参する
課題が未検証	発表の説得力が弱くなる	当事者に1人でも話を聞く

図面

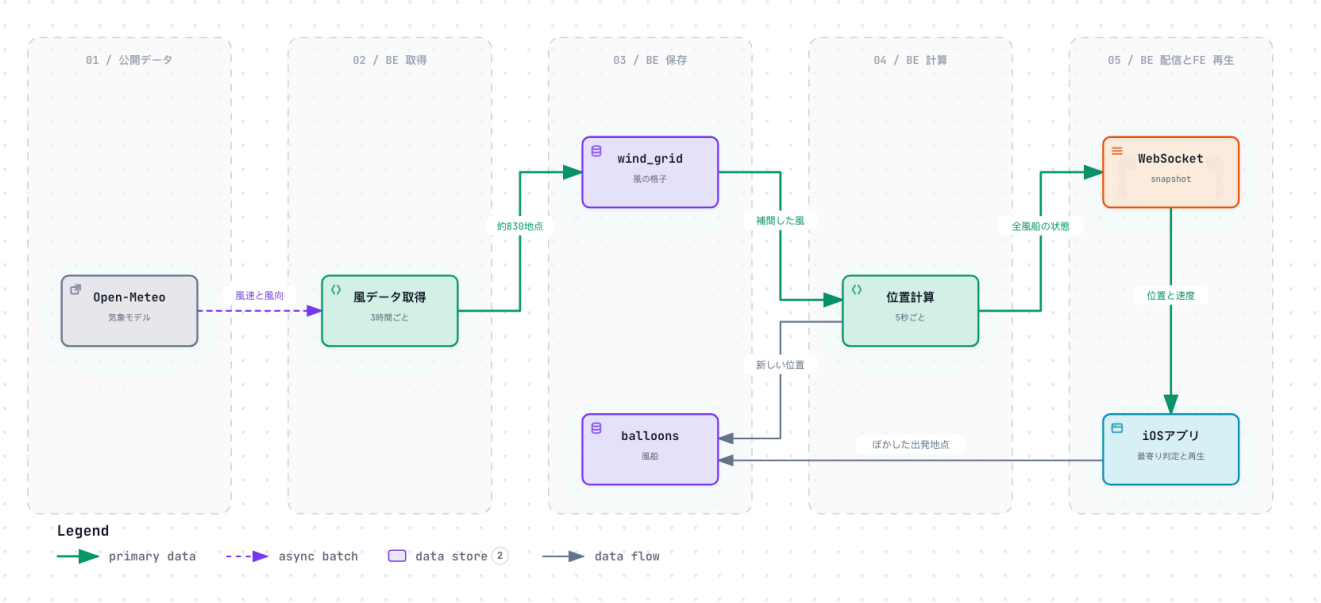
図面は全部で11枚です。シーケンス図は、ユーザー、アプリ、WebSocket、REST API、定期処理、データベース、ストレージの列で描き、使わない列は省いています。データモデルは案で、テーブル定義は実装時に確定します。

全体

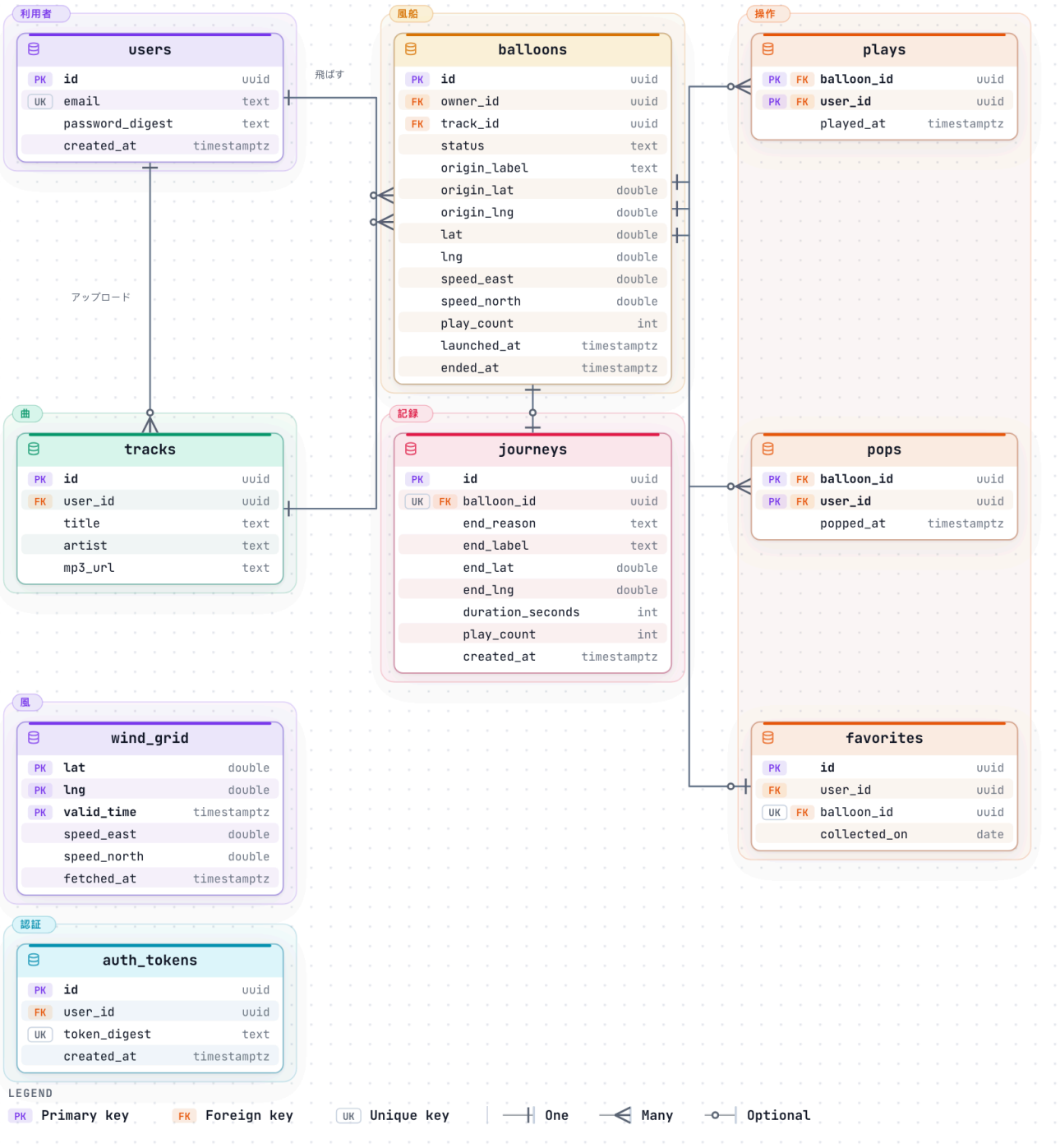
システム全体像



風データが曲になるまで

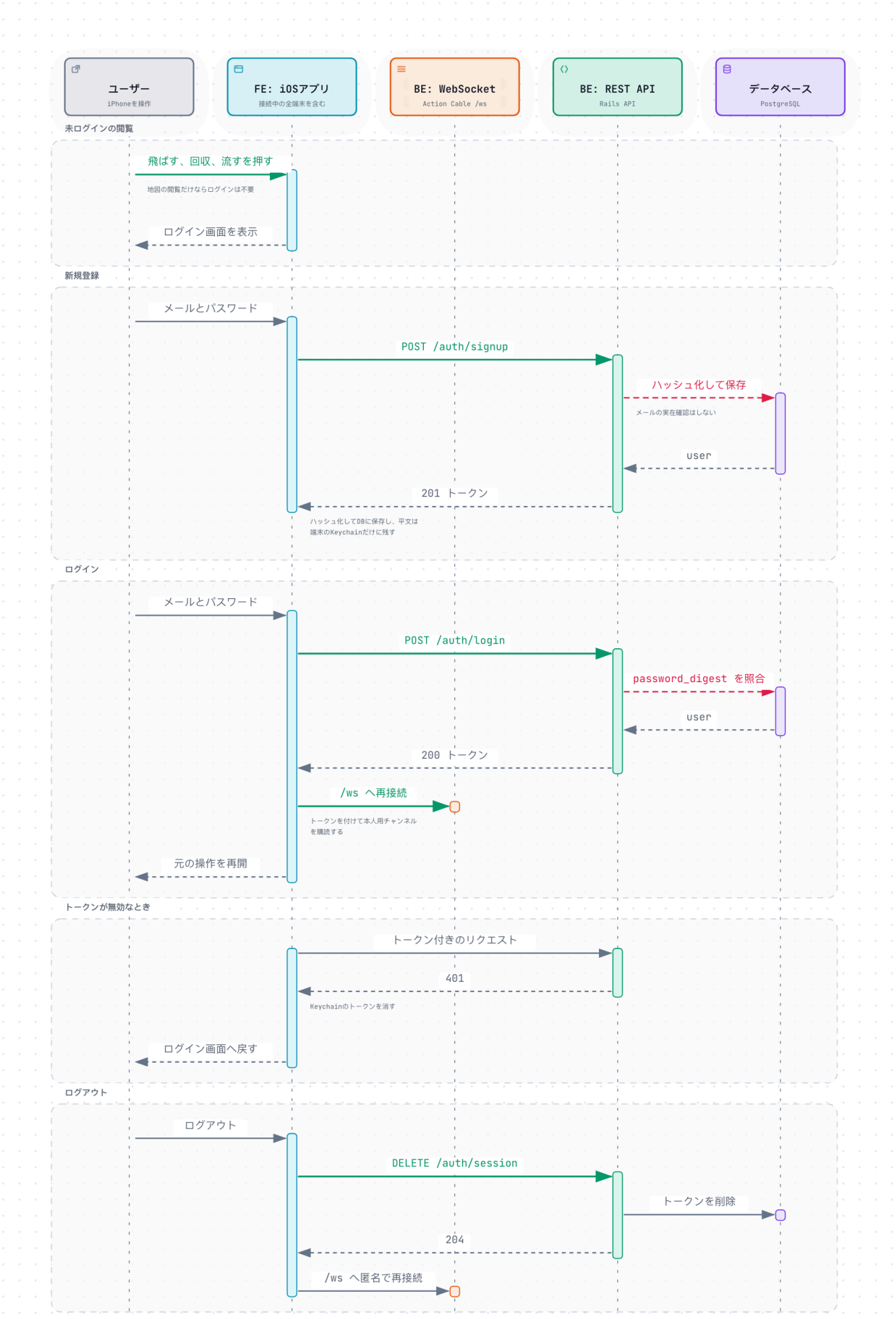

01 / 公開データ
02 / BE 取得
03 / BE 保存
04 / BE 計算
05 / BE 配信とFE 再生
Open-Meteo
気象モデル
風速と風向
風データ取得
3時間ごと
約838地点
wind_grid
風の格子
補間した風
位置計算
5秒ごと
全風船の状態
WebSocket
snapshot
位置と速度
新しい位置
balloons
風船
ぼかした出発地点
iOSアプリ
最寄り判定と再生
Legend
primary data
async batch
data store 2
data flow

データモデル案



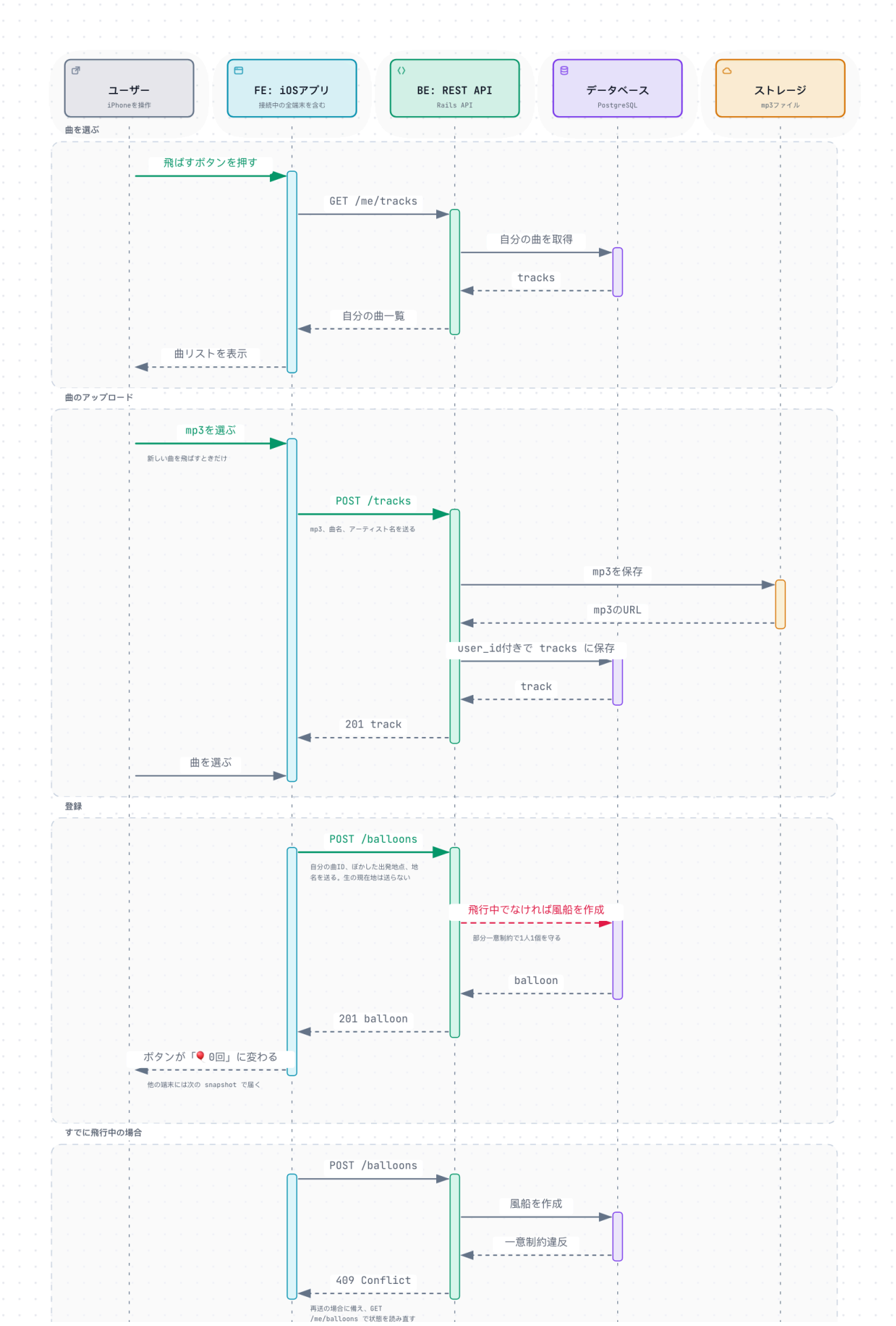
APIシーケンス

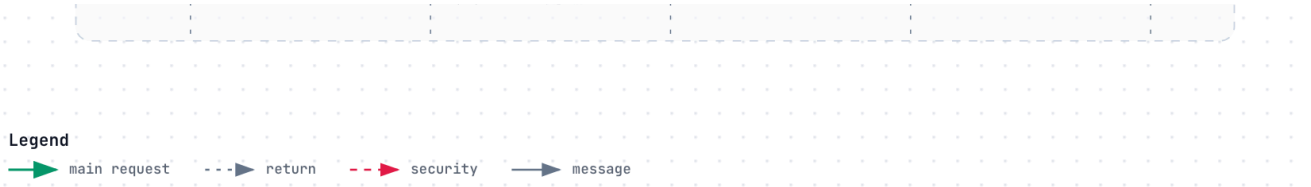
ログイン


ユーザー
iPhoneを操作
FE: iOSアプリ
接続中の全端末を含む
BE: WebSocket
Action Cable /ws
BE: REST API
Rails API
データベース
PostgreSQL
未ログインの閲覧
飛ばす、回収、流すを押す
地図の閲覧だけならログインは不要
ログイン画面を表示
新規登録
メールとパスワード
POST /auth/signup
ハッシュ化して保存
メールの実在確認はしない
user
201 トークン
ハッシュ化してDBに保存し、平文は
端末のKeychainだけに残す
ログイン
メールとパスワード
POST /auth/login
password_digest を照合
user
200 トークン
/ws へ再接続
トークンを付けて本人用チャンネル
を購読する
元の操作を再開
トークンが無効なとき
トークン付きのリクエスト
401
Keychainのトークンを消す
ログイン画面へ戻す
ログアウト
ログアウト
DELETE /auth/session
トークンを削除
204
/ws へ匿名で再接続

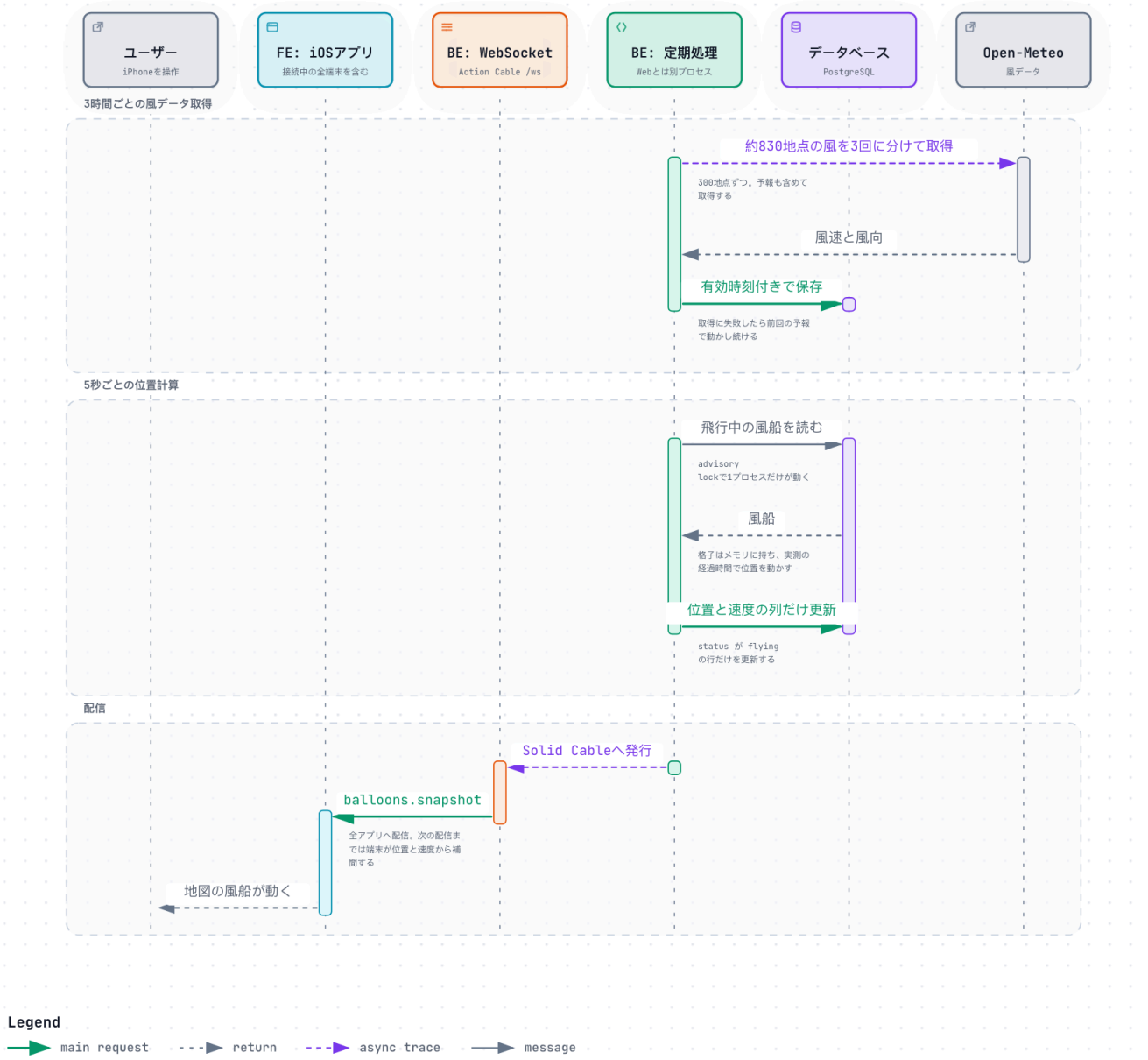


風船を飛ばす

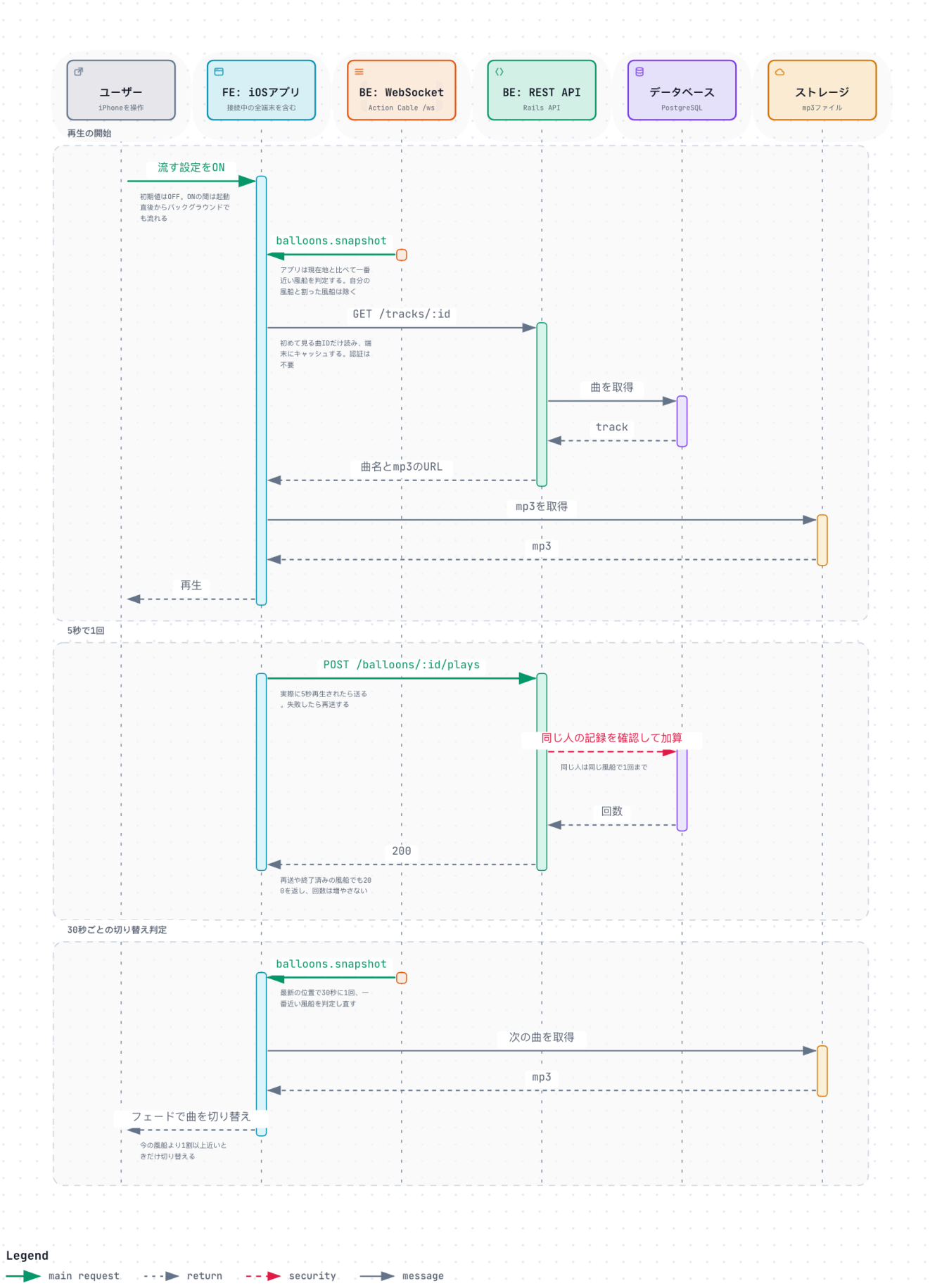

ユーザー
iPhoneを操作
FE: iOSアプリ
接続中の全端末を含む
BE: REST API
Rails API
データベース
PostgreSQL
ストレージ
mp3ファイル
曲を選ぶ
飛ばすボタンを押す
GET /me/tracks
自分の曲を取得
tracks
自分の曲一覧
曲リストを表示
曲のアップロード
mp3を選ぶ
新しい曲を飛ばすときだけ
POST /tracks
mp3、曲名、アーティスト名を送る
mp3を保存
mp3のURL
user_id付きで tracks に保存
track
201 track
曲を選ぶ
登録
POST /balloons
自分の曲ID、ぼかした出発地点、地名を送る。生の現在地は送らない
飛行中でなければ風船を作成
部分一意制約で1人1個を守る
balloon
201 balloon
ボタンが「🎈 0回」に変わる
他の端末には次の snapshot で届く
すでに飛行中の場合
POST /balloons
風船を作成
一意制約違反
409 Conflict
再送の場合に備え、GET /me/balloons で状態を読み直す



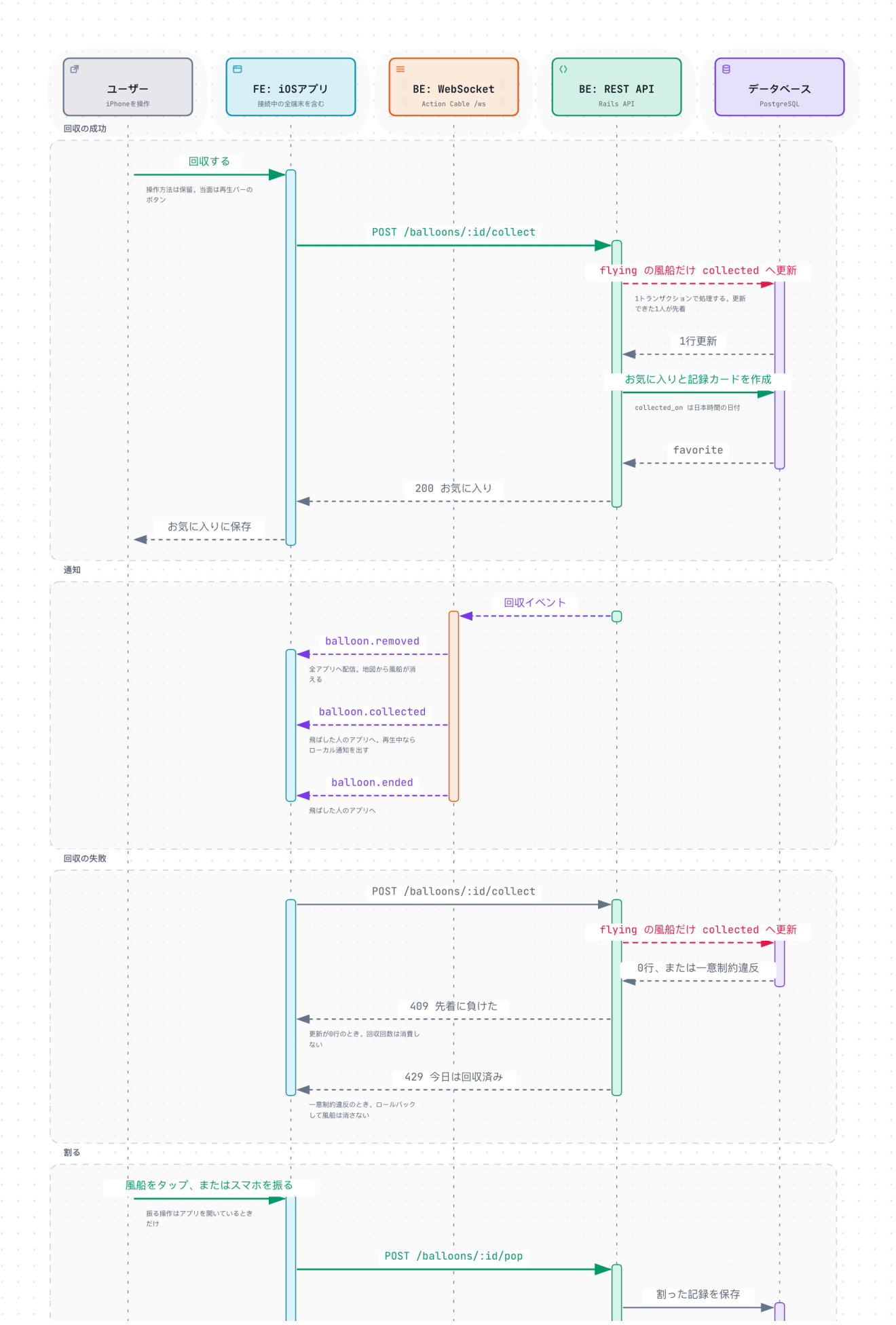
風データ取得と位置配信

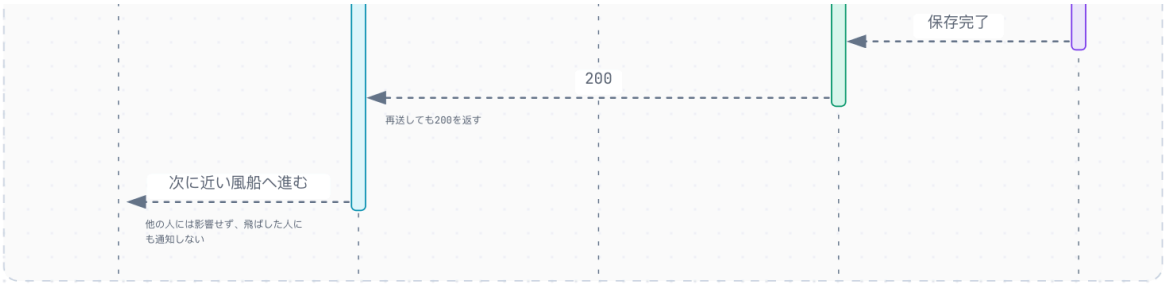


聴く、回数を数える


ユーザー
iPhoneを操作
FE: iOSアプリ
接続中の全端末を含む
BE: WebSocket
Action Cable /ws
BE: REST API
Rails API
データベース
PostgreSQL
ストレージ
mp3ファイル
再生の開始
流す設定をON
初期値はOFF。ONの間は起動直後からバックグラウンドでも流れる
balloons.snapshot
アプリは現在地と比べて一番近い風船を判定する。自分の風船と割った風船は除く
GET /tracks/:id
初めて見る曲IDだけ読み、端末にキャッシュする。認証は不要
曲を取得
track
曲名とmp3のURL
mp3を取得
mp3
再生
5秒で1回
POST /balloons/:id/plays
実際に5秒再生されたら送る。失敗したら再送する
同じ人の記録を確認して加算
同じ人は同じ風船で1回まで
回数
200
再送や終了済みの風船でも200を返し、回数は増やさない
30秒ごとの切り替え判定
balloons.snapshot
最新の位置で30秒に1回、一番近い風船を判定し直す
次の曲を取得
mp3
フェードで曲を切り替え
今の風船より1割以上近いときだけ切り替える
Legend
main request
return
security
message

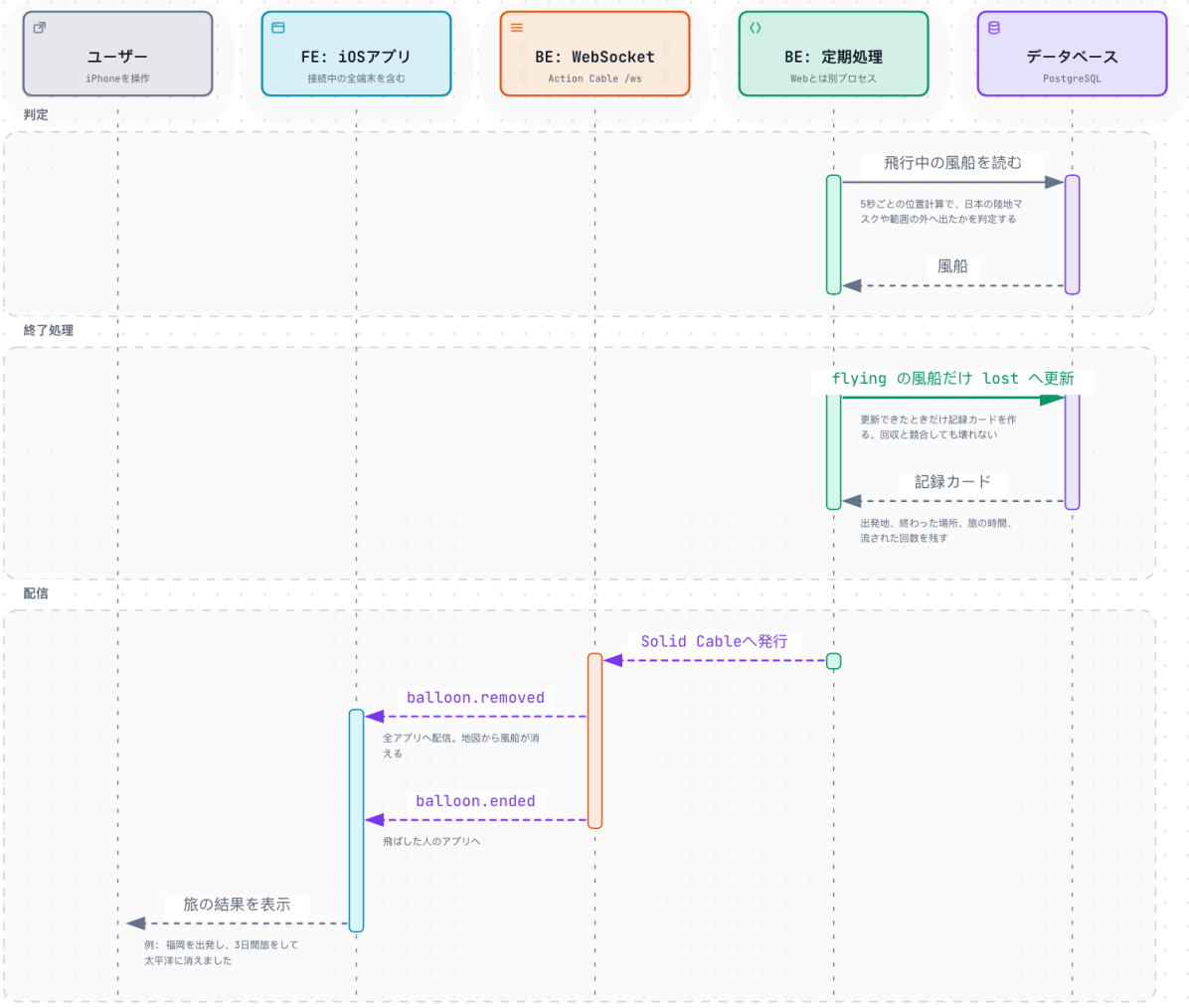
回収と割る


ユーザー
iPhoneを操作
FE: iOSアプリ
接続中の全端末を含む
BE: WebSocket
Action Cable /ws
BE: REST API
Rails API
データベース
PostgreSQL
回収の成功
回収する
操作方法は保留。当面は再生バーのボタン
POST /balloons/:id/collect
flying の風船だけ collected へ更新
1トランザクションで処理する。更新できた1人が先着
1行更新
お気に入りと記録カードを作成
collected_on は日本時間の日付
favorite
200 お気に入り
お気に入りに保存
通知
回収イベント
balloon.removed
全アプリへ配信。地図から風船が消える
balloon.collected
飛ばした人のアプリへ。再生中ならローカル通知を出す
balloon.ended
飛ばした人のアプリへ
回収の失敗
POST /balloons/:id/collect
flying の風船だけ collected へ更新
0行、または一意制約違反
409 先着に負けた
更新が0行のとき。回収回数は消費しない
429 今日は回収済み
一意制約違反のとき。ロールバックして風船は消さない
割る
風船をタップ、またはスマホを振る
振る操作はアプリを開いているときだけ
POST /balloons/:id/pop
割った記録を保存





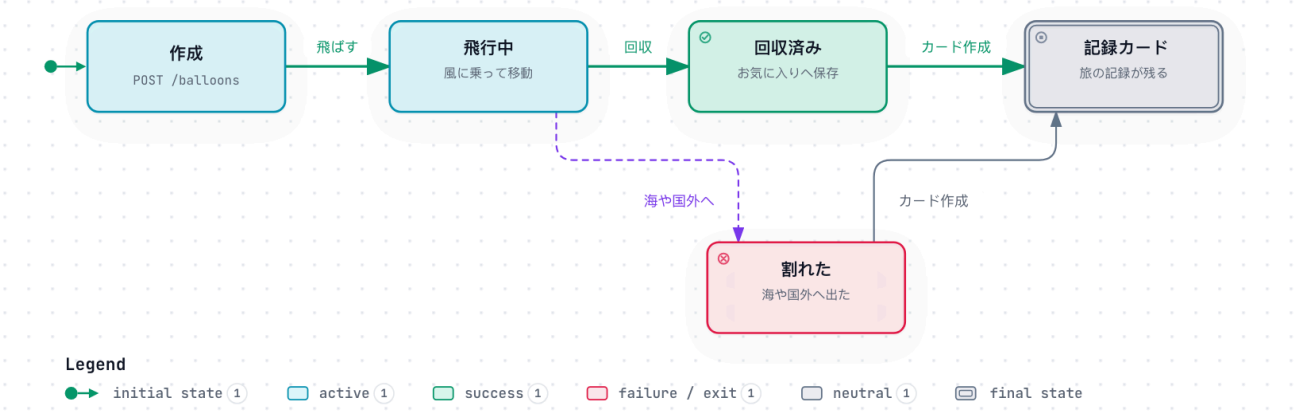
海や国外で旅が終わる



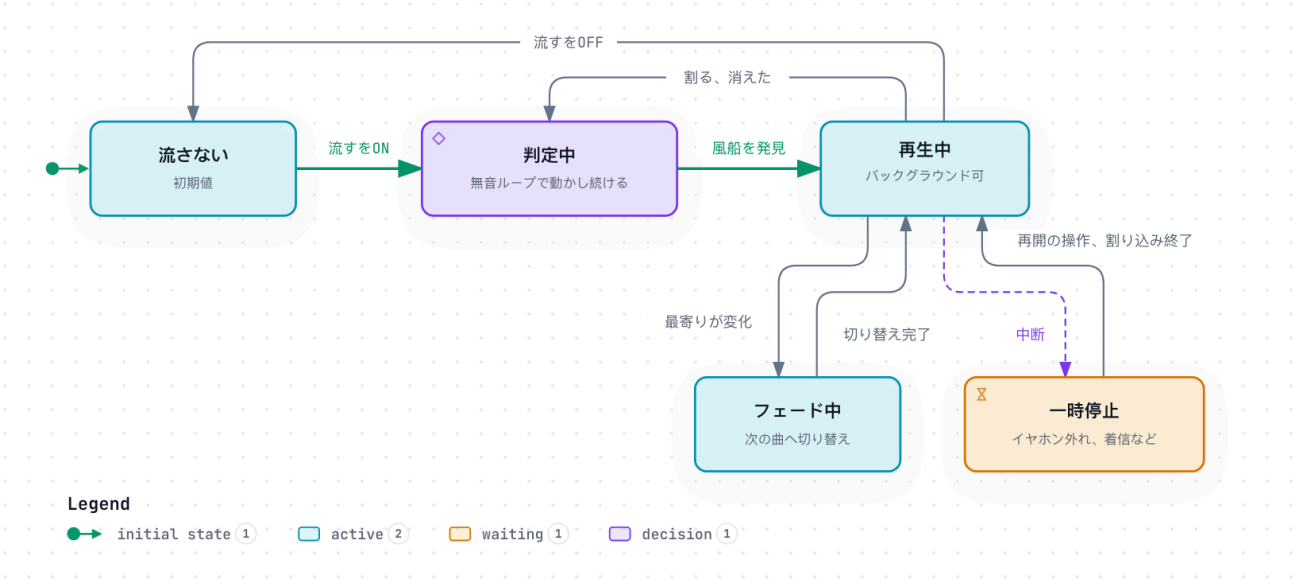


状態遷移

風船の状態遷移


作成
POST /balloons
飛ばす
飛行中
風に乗って移動
回収
回収済み
お気に入りへ保存
カード作成
記録カード
旅の記録が残る
海や国外へ
カード作成
割れた
海や国外へ出た
Legend
initial state 1
active 1
success 1
failure / exit 1
neutral 1
final state

再生の状態遷移


流すをOFF
割る、消えた
流さない
初期値
流すをON
判定中
無音ループで動かし続ける
風船を発見
再生中
バックグラウンド可
再開の操作、割り込み終了
最寄りが変化
切り替え完了
中断
フェード中
次の曲へ切り替え
一時停止
イヤホン外れ、着信など
Legend
initial state 1
active 2
waiting 1
decision 1